

s3.02 — Distributed job scheduler / orchestrator

Index

- [Interview framing](#)
- [Requirements — IC3 baseline](#)
- [Capacity and consequence — IC3 baseline](#)
- [Architecture and flow — IC3 baseline](#)
- [API and data model — IC3 baseline](#)
- [Occurrence creation and time semantics — IC3 baseline](#)
- [Attempt leases and effect idempotency — IC3 baseline](#)
- [Failure, operations, and security](#)
- [Alternatives and IC4 stretch](#)
- [Level signal — IC3 baseline vs IC4 stretch](#)
- [Interview pushback and validation](#)
- [Recall card](#)
- [References](#)

Interview framing

- Target: OCI IC3 ownership of a bounded platform component.
 - IC4 stretch changes the architecture only when a stronger requirement earns the cost.
 - These are preparation labels, not Oracle's official rubric.
- Evidence: [ranked bank](#).
 - Prior supplemental bank
 - Platform preparation, not a first-hand report.
 - This is a proposed design, not a claim about OCI internals or a reported accepted answer.
- Why this is hard
 - Naive: run cron independently on every scheduler and send each due job to a queue.
 - First break: two schedulers create the same occurrence, or a worker repeats an external effect after its lease expires.
 - Second break: a daylight-saving transition yields two or zero occurrences for one local time, and a 10-minute outage releases every missed run at once.
 - Minimum primitives
 - A durable schedule occurrence identity.
 - Transactional run creation and outbox.
 - Leased worker claims with fencing.
 - Idempotency at the downstream effect boundary.
- 🗨️ "Exactly one run record is achievable; exactly one external effect requires cooperation from the destination."
- Opening script
 - "I will separate a job definition from each scheduled occurrence and each execution attempt."
 - "I will clarify time-zone rules, missed schedules, and the downstream effect contract first."
 - "A database transaction will create each occurrence and its dispatch intent together."
 - "Workers will claim fenced attempts, while the destination deduplicates business effects by run ID."
 - "I will close with fair admission, retry timing, and recovery after a scheduler outage."

Phase	Clock	Deliverable
Intro	00:00–05:00	Establish role context and confirm the prompt
Requirements	05:00–11:00	Scope and one correctness invariant
Capacity and API	11:00–17:00	A number that forces a choice; retry contract

Phase	Clock	Deliverable
Architecture	17:00–30:00	Draw ownership and trace one request
Deep dive	30:00–45:00	Concurrency proof and a crash boundary
Operations and alternatives	45:00–50:00	Degradation, recovery, and flip condition
Wrap-up	50:00–54:00	Four-sentence recap and unresolved assumption
Interviewer questions	54:00–60:00	Team ownership and production priorities

- Clock note
 - The design slice is 45 minutes within the 60-minute round.
 - If the interviewer gives 45 minutes total, use a 2-minute intro and compress the remaining phases.
 - The interviewer's clock wins.

Requirements — IC3 baseline

- Numerical targets are interview assumptions unless an adjacent primary source states otherwise.

Question	Assumed answer	Why it changes the design
One-off or recurring?	Both; cron with explicit time zone	Need persisted UTC occurrences and schedule versions
Delivery guarantee?	At-least-once execution attempts	Business idempotency belongs at the destination
Missed schedules?	Skip by default; bounded catch-up optional	Prevents an unbounded recovery stampede
Concurrent runs of one job?	One by default	Durable admission must enforce the policy
Cancellation?	Cooperative; effects may already exist	Do not promise process termination implies rollback

- Functional scope
 - Create and version job definitions.
 - Materialize scheduled occurrences.
 - Dispatch and retry attempts.
 - Track cancellation and terminal outcomes.
 - Enforce tenant and job concurrency limits.
- Targets below are interview assumptions, not product SLAs.

Requirement	Target	Enforced by
Dispatch delay	p99 < 5 s while healthy	1 s due-index sweep and bounded queue age
Scheduling availability	99.9% regional target	Replicated metadata and multiple claimers
Default retry horizon	24 h, maximum 8 attempts	Durable nextAttemptAt and deadline
Tenant concurrency	20 admitted runs by default	Transactional slot permits
Worker lease	30 s; renew every 10 s	Lease authority and expiry reaper
Run retention	30 days hot	Partitioned run history and archival

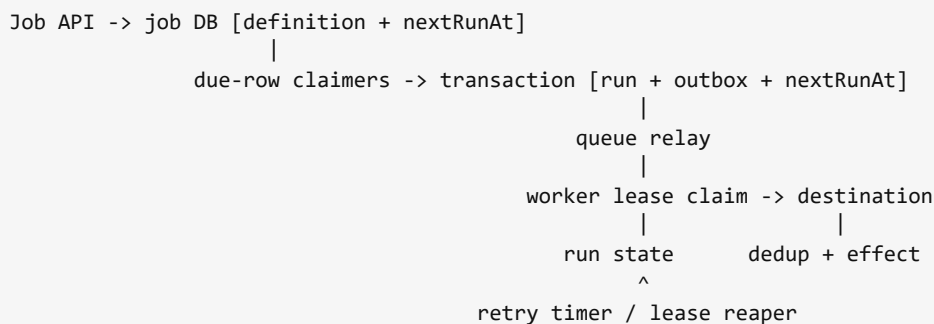
- Invariants
 - A job schedule version has at most one run for each intended UTC occurrence.
 - Each accepted run has durable dispatch intent.
 - Only the current unexpired attempt fence may commit a terminal state.
 - A destination deduplication record and business effect must commit atomically when exactly-once business application is required.
- Non-goals
 - Exactly-once arbitrary scripts.
 - Hard real-time execution.

- Unbounded catch-up after long downtime.

Capacity and consequence — IC3 baseline

- All workload values below are sizing assumptions.
- Job population
 - Assume 1M jobs averaging one run per hour.
 - $1M / 3,600 \approx 278$ runs/s on average.
 - Decision: index `nextRunAt` and claim due rows in bounded batches rather than scanning every definition.
- Worker concurrency
 - At 300 runs/s and a measured or assumed **mean** runtime of 5 s, average concurrency is 1,500.
 - A p50 runtime cannot substitute for the mean in this calculation.
 - Decision: provision from the runtime distribution and peak arrival rate, then enforce admission.
- Run storage
 - $300 \times 86,400 \times 1 \text{ KB} \approx 25.92 \text{ GB/day}$ before indexes.
 - Decision: partition history by time and retain only execution pointers in hot job rows.
- Retry storm
 - A 10-minute downstream outage can affect 180,000 scheduled runs.
 - Decision: use per-destination retry budgets and jitter, with a documented skip or catch-up policy.

Architecture and flow — IC3 baseline



- Due-row claimers
 - Buy parallel scheduling through bounded row locks.
 - Pay contention for very active jobs.
 - Run store
 - Owns occurrence uniqueness and attempt state.
 - Pays transactional write load and retention cost.
 - Queue
 - Buys buffering and worker decoupling.
 - Pays redelivery and visibility-timeout management.
 - Destination
 - Owns the business-effect deduplication boundary.
 - Pays durable run-ID retention for the supported retry horizon.
- Read due definitions through the `nextRunAt` index.
- Lock a bounded batch and verify schedule versions.
- Insert run and outbox, then advance `nextRunAt` in the same transaction.
- Claim a run attempt and tenant permit atomically.
- Send the stable run ID with the fenced attempt.
- Commit the verified outcome under the current lease and release the permit.

API and data model — IC3 baseline

- Proposed API contracts.
 - Authenticate the tenant from credentials before looking up any resource.
 - Retain idempotency records for the documented retry horizon.

Surface	Contract	Idempotency and conflicts
POST /jobs	Definition, cron/time zone or runAt, destination, retry policy	Tenant token deduplicates creation; invalid schedule is 400
PATCH /jobs/{id}	If-Match scheduleVersion	412 on stale version; edits affect future unmaterialized occurrences
POST /jobs/{id}:run	202 plus runId	Caller token creates one manual occurrence
POST /runs/{id}:cancel	Cancellation requested	Replay returns existing request; already completed effects remain
GET /runs/{id}	State, attempt, nextAttemptAt, error	Authoritative versioned status; cursor history pages

State	Key	Fields and invariant
Job	(tenant, jobId)	ScheduleVersion, cron, timeZone, nextRunAt, misfire policy
Run	(jobId, scheduleVersion, scheduledAtUTC)	RunId, state, current attempt, deadline; unique occurrence
Attempt	(runId, attemptNumber)	Fence, worker, leaseUntil, startedAt, result
Permit	(tenant, slotId)	Current run and lease; allocated with run claim
Outbox	(runId, dispatchGeneration)	Durable delivery intent
Destination dedup	(tenant, runId, effectName)	Committed outcome in the same transaction as the effect

- Reference DDL
 - Keys and uniqueness match the state table above and the [companion schema](#).
 - Queue messages, worker-local lease timers, and the destination's business tables are not relational here; only scheduler rows plus the destination dedup row are modeled.

```

CREATE TABLE jobs (
  tenant_id      text NOT NULL,
  job_id         text NOT NULL,
  shard         integer NOT NULL,          -- claimer partition for the due sweep
  schedule_version bigint NOT NULL DEFAULT 1, -- If-Match value; bumps on any schedule or misfire edit
  cron          text,                    -- NULL for a one-shot job
  run_at        timestamptz,             -- one-shot instant
  time_zone     text NOT NULL,
  destination_ref text NOT NULL,
  retry_policy  jsonb NOT NULL,          -- max attempts 8, horizon 24 h, backoff
  misfire_policy text NOT NULL DEFAULT 'SKIP',
  overlap_policy text NOT NULL DEFAULT 'SERIAL',
  next_run_at   timestamptz,
  state         text NOT NULL DEFAULT 'ACTIVE',
  create_token  text,                    -- POST /jobs idempotency
  PRIMARY KEY (tenant_id, job_id),
  UNIQUE (tenant_id, create_token),
  CHECK (state IN ('ACTIVE', 'PAUSED', 'DELETED')),
  CHECK (misfire_policy IN ('SKIP', 'COALESCE', 'CATCH_UP')),
  CHECK ((cron IS NULL) <> (run_at IS NULL)) -- exactly one schedule form
);
CREATE INDEX jobs_due ON jobs (shard, next_run_at)
  WHERE state = 'ACTIVE' AND next_run_at IS NOT NULL; -- FOR UPDATE SKIP LOCKED sweep

CREATE TABLE runs (
  tenant_id      text NOT NULL,
  job_id         text NOT NULL,
  schedule_version bigint NOT NULL,
  occurrence     text NOT NULL,          -- scheduled: intended UTC instant; manual: caller token
  run_id        text NOT NULL,
  kind          text NOT NULL,
  state         text NOT NULL DEFAULT 'READY',
  current_attempt integer NOT NULL DEFAULT 0,
  fence         bigint NOT NULL DEFAULT 0, -- increments on every attempt claim
  next_attempt_at timestamptz,
  deadline      timestamptz NOT NULL,    -- end of the retry horizon
  cancel_requested boolean NOT NULL DEFAULT false,
  created_at    timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (tenant_id, job_id, schedule_version, occurrence), -- one run per intended occurrence
  UNIQUE (tenant_id, run_id),
  FOREIGN KEY (tenant_id, job_id) REFERENCES jobs (tenant_id, job_id),
  CHECK (kind IN ('SCHEDULED', 'MANUAL', 'RECOVERY')),
  CHECK (state IN ('READY', 'RUNNING', 'RETRY_WAIT', 'SUCCEEDED', 'FAILED', 'CANCELED')),
  CHECK (current_attempt >= 0 AND fence >= 0)
);
CREATE INDEX runs_by_job ON runs (tenant_id, job_id, created_at DESC, run_id); -- history pages

CREATE TABLE attempts (
  tenant_id      text NOT NULL,
  run_id        text NOT NULL,
  attempt_number integer NOT NULL,
  fence         bigint NOT NULL,          -- equals runs.fence at claim; only this value may complete
  worker_id     text NOT NULL,
  lease_until   timestamptz NOT NULL,
  started_at    timestamptz NOT NULL DEFAULT now(),
  result        text,                    -- NULL while the attempt is live
  error         text,
  PRIMARY KEY (tenant_id, run_id, attempt_number),
  FOREIGN KEY (tenant_id, run_id) REFERENCES runs (tenant_id, run_id),
  CHECK (attempt_number >= 1),
  CHECK (result IS NULL OR result IN ('SUCCEEDED', 'FAILED', 'EXPIRED', 'CANCELED'))
);
CREATE INDEX attempts_reaper ON attempts (lease_until) WHERE result IS NULL; -- expired-lease sweep

CREATE TABLE permits (
  tenant_id      text NOT NULL,
  slot_id        integer NOT NULL,        -- pre-created rows, 20 per tenant by default
  run_id         text,                    -- NULL means free

```

```

    fence          bigint,
    lease_until timestamptz,
    PRIMARY KEY (tenant_id, slot_id),
    FOREIGN KEY (tenant_id, run_id) REFERENCES runs (tenant_id, run_id)
);
CREATE INDEX permits_free ON permits (tenant_id) WHERE run_id IS NULL;    -- claimed with SKIP LOCKED

CREATE TABLE outbox (
    tenant_id      text NOT NULL,
    run_id         text NOT NULL,
    dispatch_generation integer NOT NULL,          -- 1 at creation; +1 per durable retry intent
    deliver_after  timestamptz NOT NULL,          -- carries long retry delays the queue cannot
    payload        jsonb NOT NULL,
    published_at   timestamptz,
    PRIMARY KEY (tenant_id, run_id, dispatch_generation),
    FOREIGN KEY (tenant_id, run_id) REFERENCES runs (tenant_id, run_id),
    CHECK (dispatch_generation >= 1)
);
CREATE INDEX outbox_pending ON outbox (deliver_after) WHERE published_at IS NULL;

CREATE TABLE destination_dedup (                -- lives in the DESTINATION database, not the scheduler's
    tenant_id      text NOT NULL,
    run_id         text NOT NULL,
    effect_name    text NOT NULL,
    result         jsonb NOT NULL,                -- committed in the same transaction as the effect
    committed_at timestamptz NOT NULL DEFAULT now(),
    PRIMARY KEY (tenant_id, run_id, effect_name)
);
CREATE INDEX destination_dedup_expiry ON destination_dedup (committed_at);    -- retry-horizon purge

```

- Not captured by the DDL
 - The queue and its visibility timeout are a delivery optimization; `outbox` plus the `runs` key stay authoritative.
 - `runs` and `attempts` are partitioned by `created_at` for the 30-day hot window; the partition clauses are omitted.
 - `destination_dedup` has no foreign key because it commits in a different database than `runs`.
- Deviation from the companion
 - The companion keys `job_runs` by `(tenant_id, run_id)` and stores attempt and lease columns on the run row.
 - Here the run is keyed by its occurrence as the state table says, `run_id` is a unique alias, and `attempts` and `permits` are separate rows so the lease reaper and the admission claim lock narrow rows.

Occurrence creation and time semantics — IC3 baseline

- Schedule identity
 - Persist the intended UTC instant with the schedule version.
 - Define daylight-saving behavior explicitly for ambiguous or nonexistent local times.
 - Baseline skips nonexistent local times and runs once at the earlier occurrence of an ambiguous time.
 - Changing this policy creates a new schedule version.
 - A manual `:run` occurrence takes its identity from the caller's idempotency token, so it cannot collide with a cron instant under the same unique key.
- Due-row transaction
 - Use an index on `nextRunAt` with bounded `FOR UPDATE SKIP LOCKED` batches.
 - Recheck the locked definition and its schedule version.
 - Insert the unique occurrence and its outbox event.
 - Compute the next occurrence from the intended schedule instant, not worker completion time.
 - Commit the `nextRunAt` update with run creation.
- Missed schedules
 - Skip advances to the next future occurrence.

- Coalesce creates one explicitly labeled recovery run.
- Catch-up creates at most the configured number of occurrences per sweep.
- A job edit or cancellation races through the same job row authority.

```
MATERIALIZE(now):
  transaction:
    jobs = lock_due_rows(now, batchLimit, skipLocked=true)
    for job in jobs:
      occurrences = apply_misfire_policy(job, now, catchupLimit)
      for instant in occurrences:
        insert run(job.id, job.scheduleVersion, instant) on conflict do nothing
        if inserted: insert matching outbox
      persist job.nextRunAt using this locked scheduleVersion
```

- Queue deduplication is an optimization.
 - The database occurrence key remains correct after the queue's deduplication window expires.

Attempt leases and effect idempotency — IC3 baseline

- Claim
 - Lock the run and a free tenant permit in one transaction.
 - Require runnable state and no live attempt.
 - Increment the fencing epoch and persist the lease deadline.
- Renewal and completion
 - Renew only the matching owner and fence.
 - Complete only under the current unexpired lease and expected run state.
 - A reaper conditionally reclaims expired attempts and creates durable retry intent.
 - A delayed completion from the previous attempt is rejected.
- Physical execution limit
 - Lease expiry releases logical ownership.
 - A paused or partitioned process may still be physically running.
 - Resource-side fencing is required to stop that process from committing effects.
- Business effect
 - Use stable `runId` across every attempt.
 - In the destination database, insert the deduplication record and apply the effect in one transaction.
 - A separate dedup insert followed by a nontransactional effect has a crash gap.
 - For a third-party API, reuse its idempotency token or expose ambiguous outcome reconciliation.
- Retry timers
 - Store `nextAttemptAt` and an overall deadline durably.
 - A due index dispatches long delays through the outbox.
 - Do not assume the queue supports arbitrary delayed-delivery durations.
- Delivery authentication
 - Sign tenant, job, run, attempt, audience, timestamp, and body hash.
 - Verify the audience and replay window before applying an effect.

Failure, operations, and security

Failure	Blast radius	Detection	Degraded behavior	Recovery
Two schedulers claim one job	One occurrence	Unique-key conflict	One run survives	Re-read locked schedule state
Worker dies after effect	One run	Lease expiry	Retry may execute the handler again	Destination replays the committed run result

Failure	Blast radius	Detection	Degraded behavior	Recovery
Worker outlives lease	One run and destination	Stale completion/fence rejection	Logical retry proceeds; old process may remain	Fence effects and terminate stale process best effort
Queue outage	Dispatch backlog	Oldest outbox age	Runs remain durable	Quota-limited relay catch-up
Clock skew or DST transition	Affected schedules	Schedule audits and clock alarms	Use stored UTC occurrence policy	Recompute future instants without duplicating identities
Job DB or metadata store unavailable	All new occurrences and claims in the region	Commit failures; due-index sweep stalls	Claimed attempts keep running on their leases; no new run materializes	Restore quorum; the sweep resumes and the misfire policy bounds catch-up
Destination outage	Every run targeting it	Error rate and retry-budget exhaustion	Per-destination backoff; other destinations unaffected	Drain retries under the destination budget with jitter
Lease reaper stalls	Expired attempts are not retried	Oldest expired-lease age	Runs sit in a live-looking attempt state	Restart the reaper; conditional reclaim is idempotent
Tenant permits exhausted	One tenant	Permit wait age and rejections	That tenant's runs queue as overdue	Raise the quota or let the backlog drain; other tenants unaffected

- Observe
 - Intended-to-dispatched delay.
 - Lease expiration rate.
 - Retry amplification by destination.
 - Tenant permit saturation.
- Retry discipline
 - Exponential backoff with jitter between attempts, capped by the run's durable deadline and maximum attempt count.
 - Retry only transient destination or transport failures.
 - A validation or authorization failure marks the run failed and needs a corrective edit, not another attempt.
 - Per-destination retry budgets keep one failing destination from consuming worker capacity.
 - Every retry reuses the same `runId`, so the destination's deduplication record decides whether the effect repeats.
- Tenant isolation
 - Keep tenant permits durable rather than fail-open counters.
 - Run untrusted code in a sandbox with bounded CPU and memory.
 - Validate callback destinations and restrict egress.
 - Scope job, run, and idempotency lookups to the authenticated tenant.
 - A tenant's token receives `404` on another tenant's job or run IDs, not `403`, to avoid existence leaks.
 - Fair-share the due-index sweep so a tenant with many due jobs cannot monopolize a claim batch.
 - Dedicated execution pools keep long-running or untrusted workloads from delaying other tenants' short jobs.
- Rollout and rollback safety
 - Version payloads and schedule interpretation.
 - Shadow-compute future occurrences before switching the scheduler.
 - Drain old worker protocols before removing compatibility.
 - Canary a new claimer or worker version on one shard or tenant cohort before the fleet.
 - Occurrence uniqueness makes rollback safe: an old and a new scheduler running side by side cannot create two runs for one instant.
 - Rolling back a worker cannot un-commit a terminal state because attempt fences and run states only move forward.
 - Static stability: attempts already claimed continue under their leases while the job API or claimers deploy; only new materialization pauses.

- Schema changes follow expand, migrate, contract; a new claimer must read `nextRunAt` rows written by the previous version.

Alternatives and IC4 stretch

Option	What it is	Strength	Weakness	Choose when
SQL due index	Transactional occurrence materialization	Simple uniqueness and recovery	Hot due range at very high scale	Baseline regional scheduler
Time wheel	Bucketed in-memory timers backed by durable intent	Cheap timer scans	Recovery and rebalance complexity	Very large timer population with coarse resolution
Workflow engine	Durable multi-step orchestration	Compensation and long timers	Workflow versioning cost	Jobs evolve into branching workflows

- IC4 stretch — stronger requirement (multi-step runs)
 - Separate workflow steps and compensation identities when one run spans many effects.
- IC4 stretch — 10× load
 - Partition definitions and due indexes by tenant home cell.
 - Add fair dispatch queues per destination budget.
- IC4 stretch — cost and blast radius
 - Dedicated execution pools isolate long-running or untrusted workloads.
 - Admission rejects excess work before it turns into overdue schedules.
- IC4 stretch — migration
 - Transfer a schedule shard with an ownership epoch and a durable scan checkpoint.
 - Overlapping scans remain safe through occurrence uniqueness.
- Weakest assumption
 - The destination can atomically deduplicate the business effect.
 - Mitigation: onboard a destination only after it proves a transactional dedup record keyed by `runId`; otherwise mark its runs as uncertain on retry in the API.
- Over-engineering warning
 - Do not introduce a distributed lock per timer when a bounded indexed transaction already owns creation.

Level signal — IC3 baseline vs IC4 stretch

Phase	IC3 must land	IC4 adds	Separating signal
Scope	Define occurrence and misfire policy	Negotiate workflow semantics	Separates intent from attempt
Capacity	Use mean runtime and arrival rate	Model heavy tails and fair admission	Does not size from p50
Architecture	Job DB, due-row claimers, outbox, queue, leased workers, destination dedup with one trace	States the contention each split creates and its cost	Every component has a mechanism and a price
Correctness	Unique occurrence and fenced completion	Resource-side effect fencing	Names the destination boundary
Recovery	Durable retries and catch-up cap	Shard ownership migration	No lost timers during movement
Wrap	Recap and the downstream-idempotency tradeoff	Names the weakest assumption before being asked	Identifies the flaw first

Interview pushback and validation

- Q1: Why are duplicate queue messages harmless?
 - **IC3 baseline**

- They select the same run record and must claim its current lease.
- **IC4 stretch**
 - The destination still needs stable run-ID deduplication because attempts can overlap physically.
- Q2: The effect happened before the worker crashed?
 - **IC3 baseline**
 - Retry the same run ID and replay the destination's committed result.
 - **IC4 stretch**
 - Without destination support, expose an uncertain outcome and reconcile.
- Q3: Can a Redis TTL counter enforce the concurrency limit?
 - **IC3 baseline**
 - Only approximately if it can expire while work continues.
 - **IC4 stretch**
 - Use durable logical permits and separately fence physical effects.
- Q4: A cron fires twice during daylight-saving fallback?
 - **IC3 baseline**
 - Use the declared ambiguous-time policy and a UTC occurrence identity.
 - **IC4 stretch**
 - Version time-zone rules for reproducible scheduling audits.
- Q5 (premise reversal): Every missed run must execute, so why have a skip policy at all?
 - **IC3 baseline**
 - Use bounded catch-up and expose backlog age rather than silently skip.
 - **IC4 stretch**
 - Negotiate retention and capacity admission because an unbounded outage creates unbounded debt.
- Q6: Why not retry forever?
 - **IC3 baseline**
 - A durable deadline bounds cost and the destination deduplication horizon.
 - **IC4 stretch**
 - Use operator-authorized replay with a new explicit business intent after the original horizon.
- Q7: Why not let each worker run its own cron and skip the scheduler?
 - **IC3 baseline**
 - Every worker would fire the same occurrence, and nothing would own misfire policy, retries, or tenant admission.
 - **IC4 stretch**
 - Push only the trigger down; a worker-local timer may pre-warm, but the run row in the database stays the single occurrence authority.
- Validation scenarios
 - Race scheduler claims and verify one occurrence per schedule version and UTC instant.
 - Crash between run creation and dispatch and recover via outbox.
 - Pause a worker beyond lease expiry and reject its late completion.
 - Crash between destination dedup and effect using a real transaction boundary.
 - Exercise DST gaps, repeated times, and schedule edits during scans.
 - Recover a long outage under skip, coalesce, and bounded catch-up policies.
 - Deploy a new claimer beside the old one and verify one run per schedule version and instant.

Recall card

- "A job is a definition; a run is an occurrence; an attempt is an execution."
- "The unique occurrence key is the scheduling invariant."
- "Lease expiry does not kill a process."

- “Business idempotency lives with the business effect.”
- Wrap-up script
 - “The scheduler materializes unique occurrences and dispatch intent in one transaction.”
 - “Workers claim leased attempts under tenant admission limits.”
 - “Stable run IDs and destination transactions protect repeated business effects.”
 - “The main limits are downstream idempotency and the amount of catch-up the service can afford.”

References

- [Ranked bank](#): prompt and provenance.
- [Role framing](#): IC3 baseline, IC4 stretch, and round clock.
- [OCI role framing](#): req decode and the control-plane vocabulary that scores.
- [API/schema companion](#): related contract drills.
- Reference check: 2026-09-10.
 - Official documentation supports the mechanisms cited next to each link.
 - Capacity values and architecture choices remain proposed interview assumptions.
- [PostgreSQL SELECT locking](#): SKIP LOCKED and row-lock behavior.
- [Transactional outbox](#): durable dispatch intent.